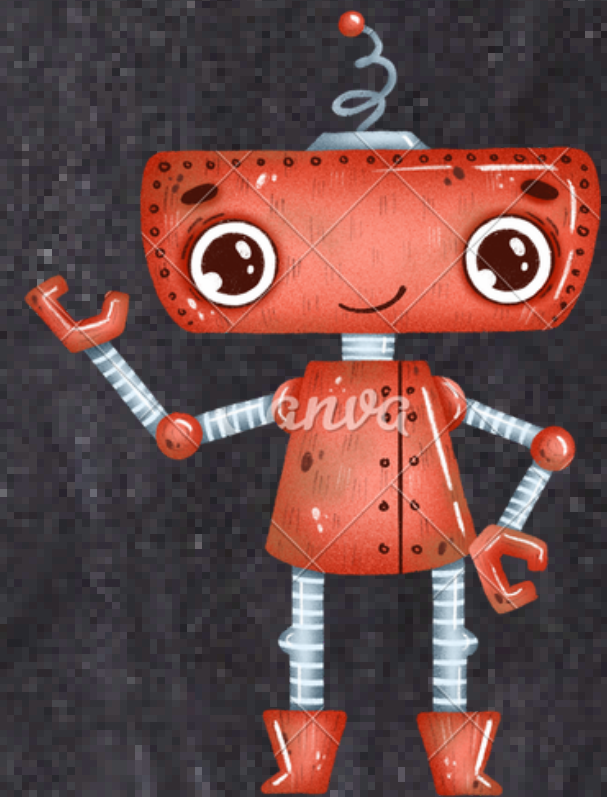
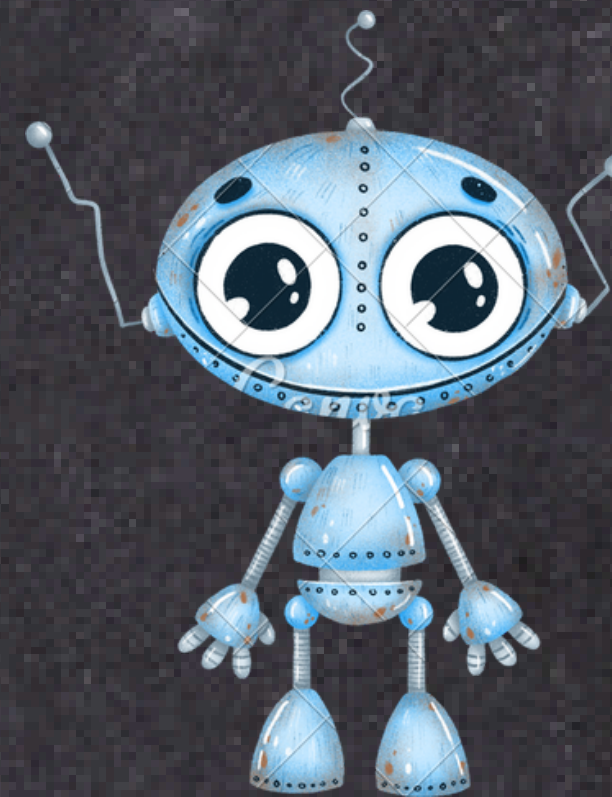
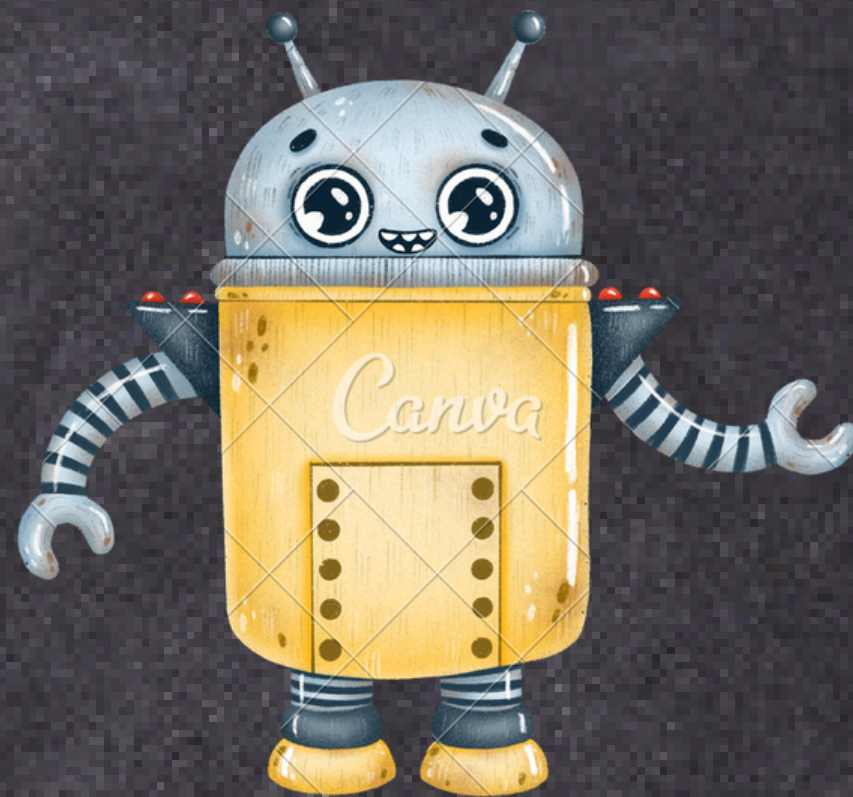
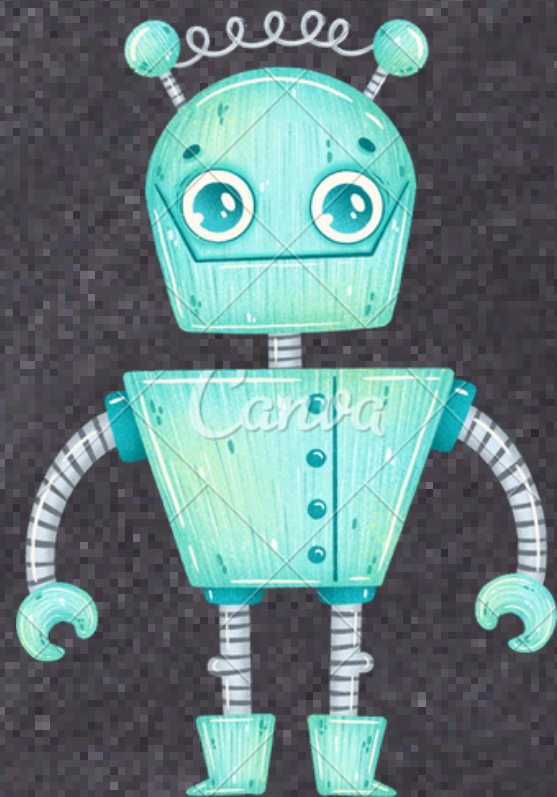


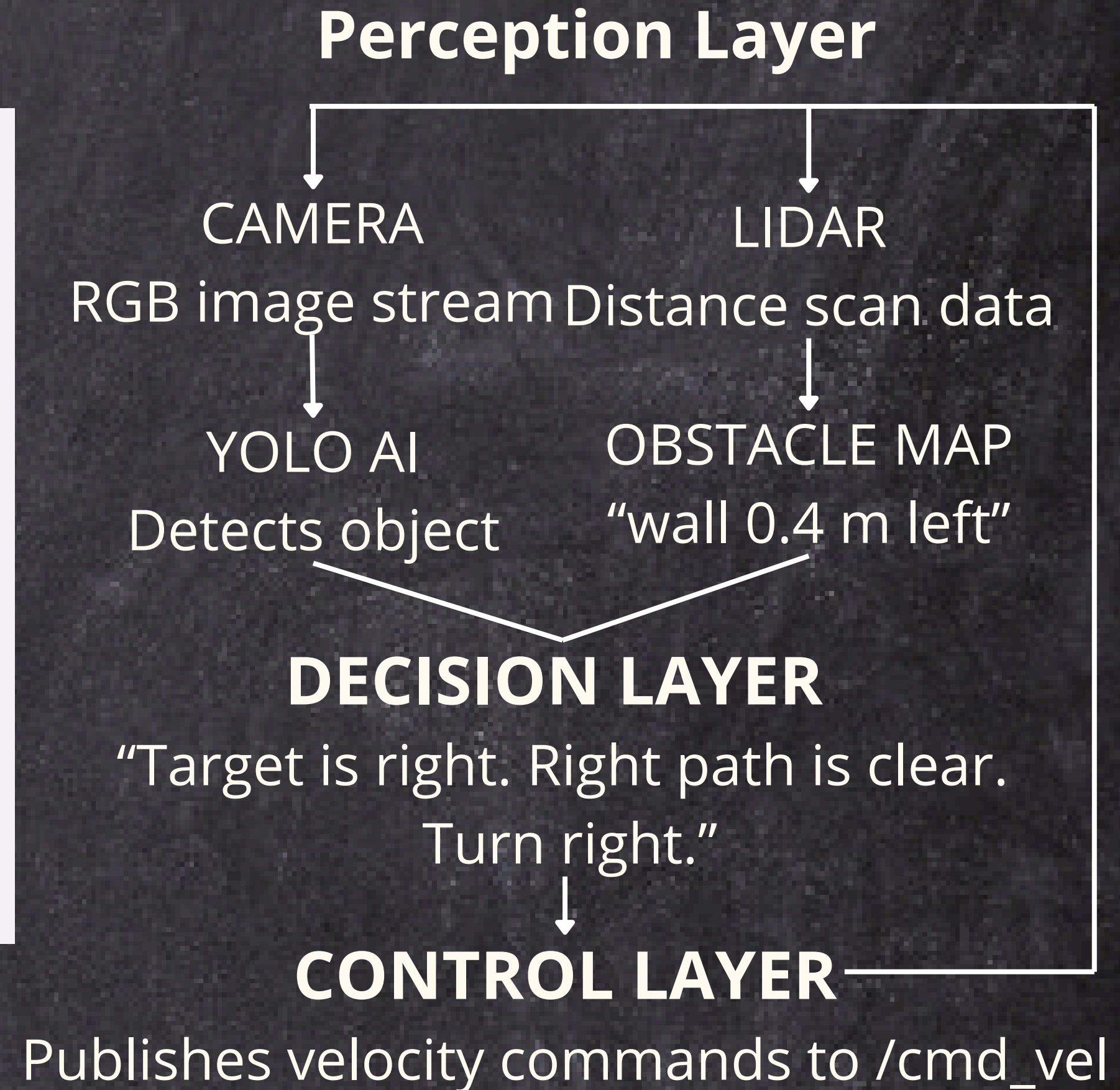
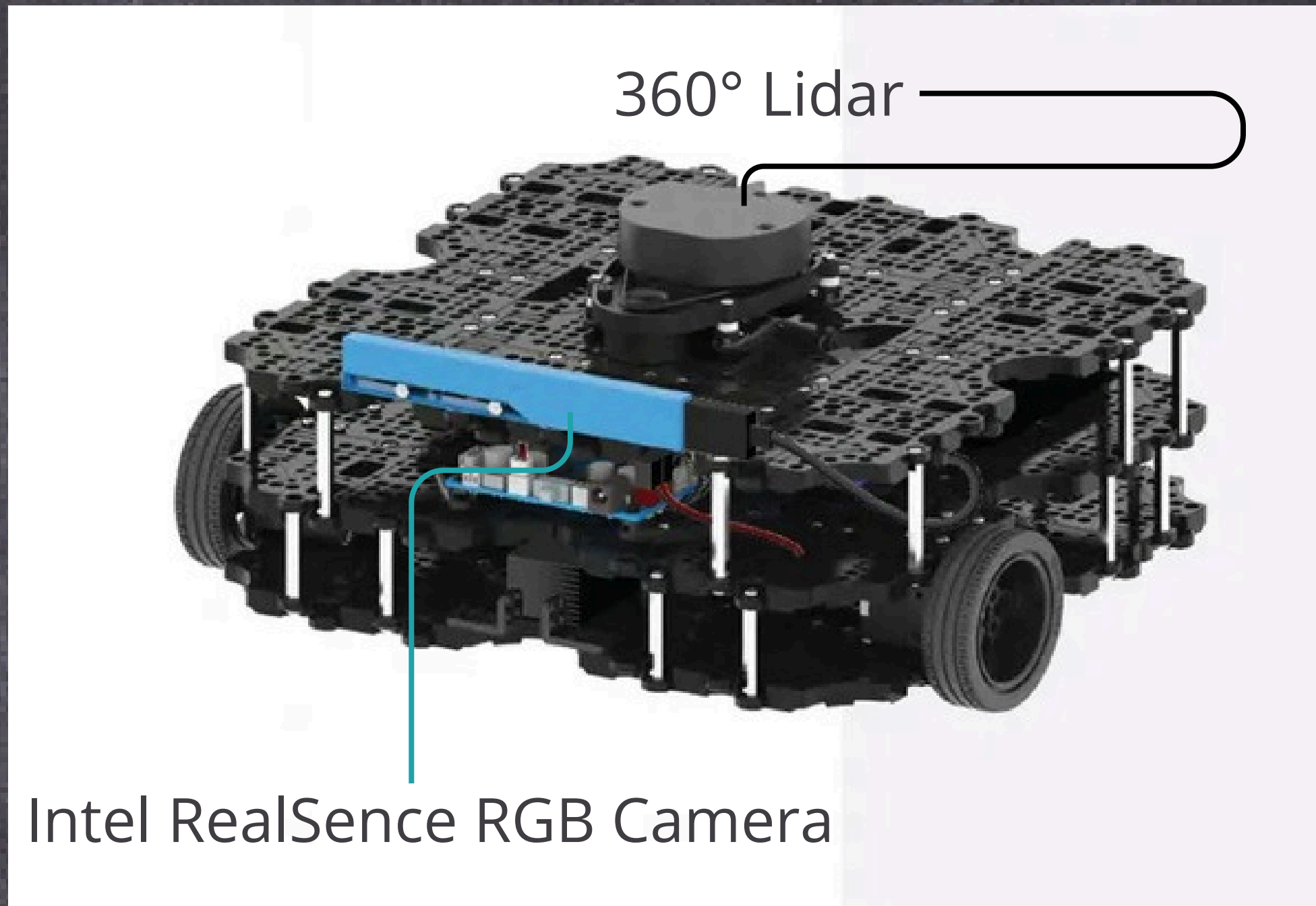
AI-BASED OBJECT FOLLOWING ROBOT USING TURTLEBOT3

PRESENTED BY :

GANESHNA BHANU BADR CHAOUCH YADLA SHIVA YANNAM SAINADH
(22529648) (12504235) (22538325) (22569272)



INTRODUCTION



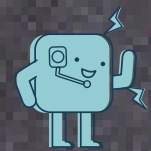
PROBLEM STATEMENT





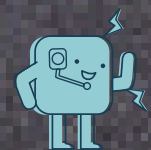
Challenge 1: FOLLOWING A MOVING TARGET

- Target moves unpredictably (no pre-planned path)
- Must maintain safe distance with the target
- Must react in real-time (e.g. 15 decisions per second)



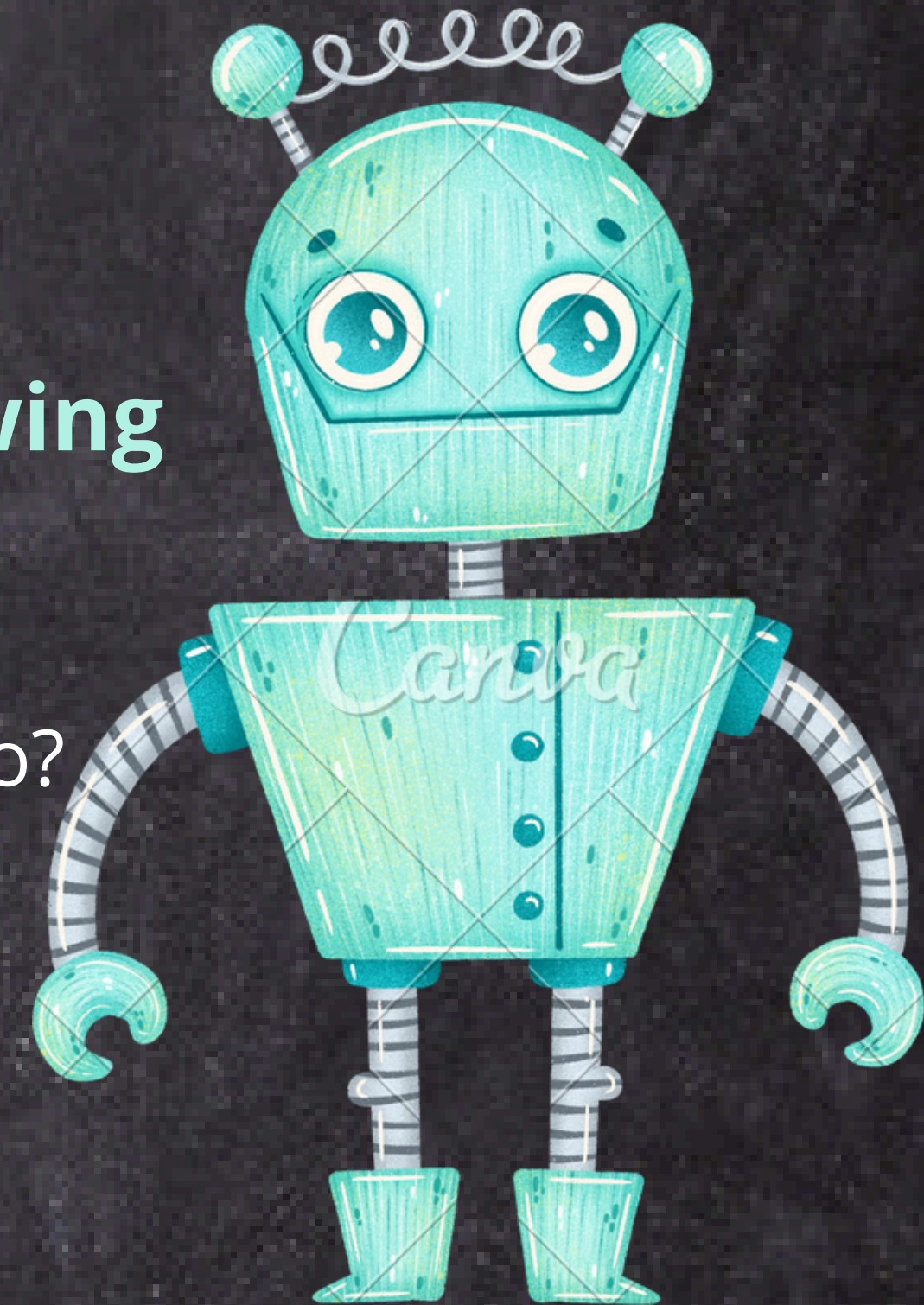
Challenge 2: Avoiding obstacles while following

- Static obstacles: walls, furniture, pillars
- Dynamic obstacles: other people, moving robots
- Conflict: target ahead but wall also ahead → what to do?



Challenge 3: Recovering when target is lost

- Target may go behind pillar or around corner
- AI may fail due to bad lighting or occlusion
- Robot must search intelligently, not just stop
- Must re-identify correct target (not a different person)



STATE OF THE ART

	Sensor(s)	Target ID Method	Robot Platform
Leceta Cuevas (2025)	Stereo Camera + Lidar	YOLO	Custom differential robot
Shalihan et al. (2023)	UWB + LiDAR	Radio Tag	TurtleBot
Huang et al. (2022)	Lidar Only	Motion Detection	Mobile robot
Mihalik et al. (2022)	Lidar Only	Motion Detection	Mobile robot
OUR PROJECT	RGB camera + Lidar	YOLO AI	TurtleBot3 Waffle

METHODOLOGY: PERCEPTION & ML TRACKING

- Use a RGB camera and YOLO on TurtleBot3 Waffle to detect the target person/object in near real time.
- Estimate the target's relative pose (distance,angle) by combining the YOLO bounding box with LiDAR range measurements.
- Extract an HSV color-histogram feature from the selected target and store it as the identity descriptor.
- For every new detection, compare its feature to the stored descriptor to track and re-identify the correct target after occlusions.
- Publish a Tracked Target message (pose + visibility state: visible, occluded, lost) at high frequency for the controller and behavior tree.

METHODOLOGY: NAVIGATION & BEHAVIOR TREE CONTROL

- Use Nav2 with LiDAR-based SLAM/localization to build a map and avoid static and dynamic obstacles while following the target.
- Convert the tracked target pose into short-horizon goal points in front of the target so Nav2 can generate safe, collision-free motion commands.
- Implement a Behavior Tree with nodes for AcquireTarget, FollowTarget, SearchTarget, ReacquireTarget, and ReturnToBase using BehaviorTree.CPP and existing TurtleBot3 BT examples.
- When the target is lost (e.g., behind a pillar), the BT triggers an intelligent local search pattern while continuously running YOLO and re-ID until the correct target is found again.
- Safety has priority: BT checks Nav2's costmaps; if the target direction is blocked by a wall or moving obstacle, the robot slows, stops, or repositions instead of blindly following, while maintaining a safe distance in real time.

TIME PLAN AND MILESTONES

08

15 April – 02 June 2026 (2nd milestone – Progress)

- Implement perception pipeline in simulation: camera → YOLO detection → target pose from LiDAR.
- Implement ML tracking and re-identification (color-histogram feature + similarity matching).
- Integrate Nav2 for SLAM + obstacle avoidance with LiDAR; basic follow behaviour working in Gazebo.
- Implement first version of Behavior Tree: AcquireTarget, FollowTarget, SearchTarget, ReacquireTarget.
- Prepare short demo video and preliminary metrics (distance error, success rate after occlusion) for the Progress presentation.

03 June – 13/17 July 2026 (Final Presentation)

- Refine perception, tracking, and BT in simulation; improve robustness in complex maps and crowded scenes.
- Run systematic simulation experiments with static and dynamic obstacles; log performance (tracking error, re-ID accuracy, near-collisions).
- Add optional features if time allows: ReturnToBase behaviour + real-robot tests.
- Prepare final 15-minute presentation and polished demo video showing the full simulation workflow and results.

18 July – 24 July 2026 (Final Report)

- Write IEEE-style report: introduction, problem statement, state of the art, methodology, experiments, discussion, future work.
- Refine figures (system diagram, BT tree, evaluation plots) and submit final code + documentation in Git .

THANK YOU

